BRNO UNIVERSITY OF TECHNOLOGY
VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY
FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF COMPUTER GRAPHICS AND MULTIMEDIA
ÚSTAV POČÍTAČOVÉ GRAFIKY A MULTIMÉDIÍ

AETHERIUM AUTOMATA: PROTOTYPE RUNTIME AND IDE INFRASTRUCTURE FOR DISTRIBUTED CONTROL SYSTEMS

AETHERIUM AUTOMATA: PROTOTYPE RUNTIME AND IDE INFRASTRUCTURE FOR DISTRIBUTED CONTROL SYSTEMS

BACHELOR'S THESIS
BAKALÁŘSKÁ PRÁCE

AUTHOR **TODO TODO**
AUTOR PRÁCE

SUPERVISOR **prof. RNDr. TODO TODO, Ph.D.**
VEDOUCÍ PRÁCE

BRNO 2026

Abstract

This thesis documents the design and a prototype implementation of tooling for distributed control systems inspired by DEVS and IEC 61499. The implemented part consists of (i) a C++ engine with a CLI that loads and validates a YAML automata description into internal data structures, (ii) an Electron+React IDE prototype with a visual state/transition editor, and (iii) an Elixir/Phoenix gateway+server infrastructure that connects the IDE via WebSockets (Phoenix Channels) and aggregates device/server status. Deterministic execution of timed automata, Lua scripting, hardware GPIO/I2C access, and device communication transports (MQTT/UDP/DDE) are not implemented in the current codebase; they are described as future work.

Abstrakt

This thesis documents the design and a prototype implementation of tooling for distributed control systems inspired by DEVS and IEC 61499. The implemented part consists of (i) a C++ engine with a CLI that loads and validates a YAML automata description into internal data structures, (ii) an Electron+React IDE prototype with a visual state/transition editor, and (iii) an Elixir/Phoenix gateway+server infrastructure that connects the IDE via WebSockets (Phoenix Channels) and aggregates device/server status. Deterministic execution of timed automata, Lua scripting, hardware GPIO/I2C access, and device communication transports (MQTT/UDP/DDE) are not implemented in the current codebase; they are described as future work.

Keywords

distributed control systems, DEVS, IEC 61499, interpreted timed automata, IoT, visual programming, deployment, monitoring, time-travel debugging

Klíčová slova

distributed control systems, DEVS, IEC 61499, interpreted timed automata, IoT, visual programming, deployment, monitoring, time-travel debugging

Reference

TODO, TODO. *Aetherium Automata: Prototype Runtime and IDE Infrastructure for Distributed Control Systems.* Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor prof. RNDr. TODO TODO, Ph.D.

Aetherium Automata: Prototype Runtime and IDE Infrastructure for Distributed Control Systems

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of the appointed supervisor. I have listed all the literary sources, publications, and other sources that were used during the preparation of this thesis. When working on the software and the LaTeX text, I used AI-assisted tooling (GitHub Copilot, GPT-5.2) for code and writing support; all technical claims about the implemented system are verified against the repository contents.

.....
TODO TODO
January 18, 2026

Acknowledgements

TODO: Add acknowledgements (supervisor, consultants, etc.).

Contents

1 Introduction **4**
1.1 Goals and Scope 4
1.2 Contributions 4
1.3 How to read this thesis 5

2 Background and Related Work **6**
2.1 DEVS (Discrete Event System Specification) 6
2.2 Timed Automata 8
2.3 Fuzzy Automata 10
2.4 Markov Automata and Probabilistic Transitions 11
2.5 Hierarchical Automata and Device Topology 13
2.6 IEC 61499 15
2.7 Visual Programming and Flow-Based Languages 16
2.8 Debugging Paradigms and Time-Travel Debugging 17
2.9 Dynamic Reconfiguration and Online Changes 18
2.10 YAML-Based Modeling and Specification 20
2.11 ROS2 and Micro-ROS 21
2.12 Comparison: Node-RED vs. 4diac/FORTE vs. Aetherium Automata 22

3 Requirements and System Design **24**
3.1 Assignment requirements vs. repository status 24
3.2 High-level architecture 25
3.3 Automata representation 25

4 Implementation **26**
4.1 C++ engine (RTE prototype) 26
4.2 Gateway and server infrastructure (Elixir/Phoenix) 27
4.3 IDE prototype (Electron + React) 27

5 Demonstration **29**
5.1 Runnable demonstrator (current) 29
5.2 Planned end-to-end case study (not implemented yet) 29

6 Discussion and Future Work **30**
6.1 Architectural trade-offs 30
6.2 Positioning relative to Node-RED and 4diac 30

7 Conclusion **31**

Bibliography **32**

A Repository Overview **33**

B Build and Run Instructions **34**
B.1 Engine (C++) 34
B.2 Gateway (Elixir/Phoenix) 34
B.3 Server (Elixir agent) 34
B.4 IDE (Electron + React) 34

C Notes on Current Limitations **35**

List of Figures

3.1 Schematic view of the implemented communication/control plane and the current engine prototype boundary. 25

4.1 A simplified message-flow sketch of the implemented device aggregation path. 28
4.2 A schematic view of the IDE panels to complement the text description. . . 28

Chapter 1

Introduction

Distributed control systems appear across industrial automation, robotics, and IoT. In practice, they combine hard constraints (limited memory, real-time behavior, high reliability) with a high degree of heterogeneity (microcontrollers, Linux nodes, gateways, cloud services). This creates a recurring engineering problem: even when the control logic can be expressed as state machines or function blocks, it is difficult to *model*, *deploy*, and *debug* the resulting distributed system in a reproducible way.

This thesis documents **Aetherium Automata**, a project that aims to combine:

- a formal foundation for modeling (DEVS and IEC 61499-inspired event-driven automation),
- a lightweight runtime environment (RTE) for embedded and Linux targets,
- an IDE and infrastructure for visual modeling, deployment, monitoring, and time-travel debugging.

1.1 Goals and Scope

The assignment requires studying DEVS, IEC 61499, Node-RED, Eclipse 4diac/FORTE, ROS2 and Micro-ROS, and designing a distributed RTE and an IDE. The repository contains both design documentation and an initial implementation across three layers:

- **C++ engine (RTE prototype):** YAML parsing into internal structures, a minimal CLI, and basic validation.
- **Infrastructure:** an Elixir/Phoenix gateway and server that communicate over Phoenix Channels.
- **IDE prototype:** an Electron+React application containing a visual automata editor and a gateway monitoring/control panel.

This thesis therefore focuses on *explaining the modeling foundations and the architecture* needed to build such a system, and uses the repository prototype to ground those decisions.

1.2 Contributions

The primary contribution of this work is a **repository-backed, end-to-end prototype skeleton**:

- a reproducible YAML-to-model pipeline in C++ (parser + validator),
- a working WebSocket control plane between IDE and gateway (Phoenix Channels),
- a visual editor for states and transitions suitable for iterating on the automata representation.

1.3 How to read this thesis

Chapter 2 summarizes the relevant theory and existing toolchains. Chapter 3 maps the assignment requirements to a concrete architecture and clearly states what is implemented. Chapter 4 describes the codebase as it exists. Chapter 5 presents the demonstrator that can be run from the repository. Chapter 6 enumerates the missing parts and an implementation roadmap.

Chapter 2

Background and Related Work

This chapter provides the theoretical and practical context needed to understand the design. The structure follows three main themes: first, the *mathematical foundations* that provide formal semantics for modeling and verification (DEVS, timed automata, IEC 61499); second, the *workflow and system features* that enable practical development, debugging, and deployment (visual programming, time-travel debugging, dynamic reconfiguration, and YAML-based specification); and third, the *related work and positioning* that shows how this project relates to existing middleware (ROS2) and engineering tools (Node-RED, 4diac/-FORTE).

The goal is to establish both the theoretical rigor and the practical engineering patterns that shape the architecture described in Chapter 3.

2.1 DEVS (Discrete Event System Specification)

DEVS, formalized by Zeigler, is a mathematical framework for modeling and simulation of discrete-event systems [6]. The assignment requires studying DEVS because it provides a rigorous foundation for hierarchical composition and time-aware execution.

2.1.1 Formalism: Atomic DEVS

An atomic DEVS model is a tuple:

$$M = \langle X, Y, S, \delta_{ext}, \delta_{int}, \delta_{con}, \lambda, ta \rangle$$

where:

- X is the set of input events,
- Y is the set of output events,
- S is the set of sequential states,
- $\delta_{ext} : Q \times X \to S$ is the external transition function (where $Q = \{(s, e) \mid s \in S, 0 \leq e \leq ta(s)\}$ is the total state including elapsed time),
- $\delta_{int} : S \to S$ is the internal transition function,
- $\delta_{con} : Q \times X \to S$ is the confluent transition function (resolves simultaneous internal and external events),

- $\lambda : S \to Y$ is the output function,
- $ta : S \to \mathbb{R}_0^+ \cup \{\infty\}$ is the time-advance function.

The execution semantics are as follows:

1. The model remains in state s for time $ta(s)$.
2. If no external event arrives during that interval, an internal transition fires: the model produces output $\lambda(s)$, then transitions to $\delta_{int}(s)$.
3. If an external event x arrives at elapsed time $e < ta(s)$, the model transitions to $\delta_{ext}((s, e), x)$ without producing output.
4. If both occur simultaneously (elapsed time equals $ta(s)$ and an external event arrives), the confluent transition δ_{con} is invoked.

This formalism explicitly models *when* a system evolves, not just *how*. This temporal explicitness is critical for deterministic simulation and for real-time embedded execution.

2.1.2 Coupled DEVS and closure under coupling

A coupled DEVS model is a tuple:

$$N = \langle X, Y, D, \{M_d\}, \{I_d\}, \{Z_{i,d}\} \rangle$$

where:

- D is the set of component names,
- M_d is the DEVS model (atomic or coupled) for each $d \in D$,
- I_d is the set of influences for component d (which other components can send it events),
- $Z_{i,d}$ is the translation function mapping outputs of component i to inputs of component d.

The key property is *closure under coupling*: a coupled model can itself be viewed as an atomic model with equivalent behavior. This enables hierarchical composition: subsystems can be encapsulated and reused as black-box components.

For an IDE, this maps directly to:

- Atomic models → primitive editor blocks (states, transitions, function blocks).
- Coupled models → hierarchical subsystems (grouped components with defined interfaces).
- Reusable libraries → catalogs of pre-validated coupled models.

While Aetherium Automata does not yet implement hierarchical execution, the architecture and representation are shaped by this distinction: the design anticipates reusable components and an explicit interface (inputs/outputs) per unit.

2.1.3 Why DEVS matters for this project

DEVS is valuable as a *composition formalism*: coupled models define hierarchical systems from atomic parts, which is aligned with the assignment requirement for hierarchical block schematics in the style of DEVS.

The most relevant aspect for this project is that DEVS makes time an explicit part of the semantics. In other words, a model does not just react to events; it also defines when it will evolve autonomously (via ta and δ_{int}). This property is a good fit for building a runtime that can be both (a) executed on real targets with real-time constraints and (b) simulated deterministically for debugging.

2.1.4 Atomic vs. coupled models

DEVS distinguishes *atomic* behavior (a transition system with explicit time advance) from *coupled* behavior (a hierarchical composition of submodels with explicit coupling). This separation is useful for an IDE-driven workflow:

- Atomic models map naturally to an editor primitive (a state machine or function-block-like component).
- Coupled models map naturally to hierarchical schematics (subsystems, sub-automata, reusable building blocks).

Illustrative example (conceptual). An atomic DEVS model for a timer might define an internal transition that fires after a delay $ta(s) = T_{timeout}$, and an external transition that resets the timer when an input event arrives before the timeout. This "scheduled internal step + interrupting external input" pattern is exactly the kind of behavior the assignment expects from an interpreted runtime.

2.1.5 DEVS simulation protocol and determinism

If the semantics define a unique next event time and a deterministic transition decision, then a run can be reproduced from an initial snapshot and the same input event stream. This is the core requirement for trace-based debugging and replay: the trace is meaningful only if execution decisions are repeatable.

In a DEVS simulator, the executive maintains an event calendar sorted by time-of-next-event for each component. The simulator advances simulation time by selecting the component(s) with the smallest time-of-next-event, invoking their internal transitions, propagating outputs through couplings, and invoking external transitions in recipient components. This execution loop is deterministic: given the same initial state and input trajectory, the simulator will produce the same sequence of state changes.

This determinism is a design requirement for time-travel debugging: if we record state snapshots and input events, we can replay execution forward or backward to inspect system behavior at any point in time.

2.2 Timed Automata

The assignment mentions *timed automata*, and timed execution semantics are a central design concern for deterministic automation. Timed automata extend finite-state machines

with real-valued clocks and timing constraints (guards) on transitions; the canonical formalization is due to Alur and Dill [2].

2.2.1 Formalism

A timed automaton is a tuple:

$$A = \langle L, L_0, C, \Sigma, E, I \rangle$$

where:

- L is a finite set of locations (states),
- $L_0 \subseteq L$ is the set of initial locations,
- C is a finite set of clocks (real-valued variables that increase at rate 1),
- Σ is a finite alphabet (input symbols),
- $E \subseteq L \times \Sigma \times \Phi(C) \times 2^C \times L$ is the set of edges, where each edge is a tuple $\langle \ell, a, \varphi, \lambda, \ell' \rangle$ meaning: from location ℓ on input a, if guard φ is satisfied, reset clocks in λ and move to ℓ',
- $I : L \to \Phi(C)$ is a mapping from locations to invariants (constraints that must hold while in that location).

Here $\Phi(C)$ is the set of clock constraints (boolean combinations of comparisons like $x \leq 5$ or $x - y > 3$).

2.2.2 Semantics and reachability

The semantics of timed automata are defined by a transition system over configurations (l, v) where $l \in L$ is the current location and $v : C \to \mathbb{R}_{\geq 0}$ is a clock valuation.

Two types of transitions are possible:

1. **Delay transitions:** $(l, v) \xrightarrow{d} (l, v + d)$ if the invariant $I(l)$ is satisfied throughout the interval $[0, d]$.
2. **Discrete transitions:** $(l, v) \xrightarrow{a} (l', v')$ if there exists an edge $\langle l, a, \varphi, \lambda, l' \rangle$ such that $v \models \varphi$, and v' is obtained by resetting clocks in λ to 0 and leaving others unchanged.

The reachability problem for timed automata asks: given an automaton and a target location, is there a sequence of transitions that reaches it? Alur and Dill proved that this problem is decidable by constructing a finite quotient of the infinite state space using *regions* (equivalence classes of clock valuations with the same qualitative behavior).

2.2.3 Zones and symbolic execution

In practice, verification tools use *zones* (convex unions of regions represented as Difference Bound Matrices) to efficiently explore the reachable state space. This is the foundation of tools like UPPAAL, which performs model checking and simulation of timed automata networks.

For a runtime implementation, timed automata semantics provide:

- A formal model for expressing timing requirements (deadlines, timeouts, periodic behavior).
- A deterministic execution model (given a clock valuation and input event, the next state is uniquely determined).
- A basis for verification: properties like "the system responds within 100ms" can be checked against the automaton model.

The repository prototype currently focuses on representation and validation, while this section provides the vocabulary that shapes the runtime loop: clocks, deadlines, and deterministic scheduling decisions. In practical terms, a timed automata runtime needs (at minimum) a representation of clocks, a way to evaluate guards, and a scheduler that decides when to take an internal step or fire a deadline.

This matters for the design chapters because timing semantics affect the entire toolchain: validation rules, what the IDE must represent, and what kinds of traces can be recorded.

2.3 Fuzzy Automata

Real-world IoT systems often operate with uncertain or imprecise information. Traditional finite state machines use crisp Boolean transitions (a condition is either true or false), but sensor readings, environmental conditions, and system states are frequently approximate. Fuzzy automata extend state machines with fuzzy logic to handle this uncertainty gracefully.

2.3.1 Fuzzy membership and transition guards

In a fuzzy automaton, transition guards evaluate to a degree of truth in $[0, 1]$ rather than simply true/false. For example, a temperature sensor reading might be "somewhat high" (membership 0.7) rather than strictly above/below a threshold.

Formally, a fuzzy guard g maps system state to $[0, 1]$:

$$g : S \to [0, 1]$$

Common membership functions include:

- **Trapezoidal:** gradual transition between regions (e.g., temperature "low", "medium", "high").
- **Gaussian:** smooth bell curves for continuous phenomena.
- **Triangular:** simpler approximations with linear slopes.

When multiple guards are partially satisfied simultaneously, the system can:

1. **Fire the strongest:** select the transition with maximum membership degree.
2. **Weighted combination:** execute actions proportionally to membership degrees.
3. **Threshold:** fire all transitions above a minimum membership (e.g., > 0.5).

2.3.2 Application to IoT control

Fuzzy automata are particularly useful for:

- **Sensor fusion:** combining noisy readings from multiple sensors.
- **Adaptive thresholds:** smooth state transitions without hysteresis artifacts.
- **Human-like reasoning:** "if temperature is somewhat high and humidity is very high, then...".

The YAML specification supports fuzzy guards through Lua scripting:

```
function fuzzyHigh(temp)
  if temp < 80 then return 0
  elseif temp > 100 then return 1
  else return (temp - 80) / 20  -- linear ramp
  end
end

function condition()
  local t = value("temperature")
  return fuzzyHigh(t) > 0.6  -- fire if membership > 0.6
end
```

This enables control logic that degrades gracefully under uncertainty rather than exhibiting abrupt mode changes.

2.4 Markov Automata and Probabilistic Transitions

While fuzzy logic handles *uncertain state observations*, probabilistic automata model *non-deterministic behavior* where multiple outcomes are possible from the same state. This is essential for modeling unreliable components, fault scenarios, and adaptive systems.

2.4.1 Discrete-time Markov chains

A Markov automaton extends a finite state machine with probability distributions over transitions. Given state s and input a, instead of a single target state, the system transitions to state s' with probability $p(s' \mid s, a)$.

Formally:

$$\sum_{s' \in S} p(s' \mid s, a) = 1$$

This models scenarios like:

- Unreliable actuators: 90% success rate, 10% failure.
- Network packet loss: message arrives with 95% probability.
- Random exploration: choose action A with 70%, action B with 30%.

2.4.2 Implementation in YAML

The YAML format supports probabilistic transitions via the `weight` field:

```
transitions:
  ToSuccess:
    from: Attempting
    to: Success
    weight: 0.9
  ToFailure:
    from: Attempting
    to: Failure
    weight: 0.1
```

At runtime, the engine normalizes weights to probabilities and samples accordingly. For debugging and verification, traces record which branch was taken, enabling replay even for stochastic systems.

2.4.3 Verification challenges and solutions

Probabilistic model checking asks questions like:

- "What is the probability of reaching an error state within 100 steps?"
- "Is the expected time to complete the task bounded?"

Tools like PRISM can analyze Markov models, but require discrete state spaces. For Aetherium, planned integration involves:

1. Export automaton to PRISM/UPPAAL format.
2. Run probabilistic model checking offline.
3. Annotate YAML with verified bounds (e.g., "success probability ≥ 0.99").

This combines the ergonomics of visual design with the rigor of formal verification.

2.4.4 Extension: Petri nets for concurrent workflows

While automata excel at sequential control, *Petri nets* are better suited for modeling concurrent resource allocation and dataflow. A Petri net consists of places (states), transitions (events), and tokens (resources). Unlike automata (one active state), Petri nets allow multiple places to hold tokens simultaneously, naturally expressing parallelism.

Potential use cases:

- Multi-resource scheduling (e.g., multiple sensors feeding a single processor).
- Pipeline parallelism (data flowing through multiple processing stages).
- Deadlock analysis (reachability of states where no transitions can fire).

The YAML format could be extended to support Petri net semantics alongside automata, or Petri nets could be compiled to equivalent hierarchical automata for execution. This is documented as a future extension (Appendix **??**).

2.5 Hierarchical Automata and Device Topology

A key architectural feature is support for *nested automata*: complex systems are built by composing smaller, validated components. This maps directly to DEVS coupled models and enables modular design, reusability, and distributed deployment.

2.5.1 Composition and black-box interfaces

A composite automaton is an automaton whose states can themselves contain sub-automata. From the outside, it exposes:

- **Input ports:** external events that trigger internal behavior.
- **Output ports:** events emitted by internal states.
- **Variables:** shared state visible to parent or opaque to parent.

Internally, the composite contains:

- A set of child automata (atomic or composite).
- Event routing rules: how parent inputs map to child inputs, and child outputs to parent outputs.
- Local state machines that orchestrate children.

This enables designs like:

```
TemperatureController (composite)
  +-- SensorFusion (atomic): reads multiple sensors
  +-- FuzzyLogicController (atomic): computes setpoint
  +-- ActuatorDriver (composite)
      +-- PWMGenerator (atomic)
      +-- FaultDetector (atomic)
```

2.5.2 Device mapping and distributed execution

In a distributed IoT system, different automata (or sub-automata) may execute on different physical devices. The deployment model specifies:

- **Automaton-to-device mapping:** which automaton runs where.
- **Communication bindings:** how events cross device boundaries (MQTT topics, DDS, UDP).
- **Resource constraints:** memory/CPU limits per device.

Example deployment:

```
devices:
  - id: esp32_sensor
    automata: [SensorFusion]
    transport: mqtt
  - id: rpi_gateway
```

```
    automata: [FuzzyLogicController, ActuatorDriver]
    transport: mqtt+dds
  - id: esp32_actuator
    automata: [PWMGenerator]
    transport: mqtt
```

The IDE validates that:

1. All event connections between automata have valid transport bindings.
2. Device memory constraints are satisfied (automaton size < device capacity).
3. Network topology is reachable (devices can communicate).

2.5.3 Dynamic re-mapping and migration

At runtime, automata can be migrated between devices:

- **Load balancing:** move heavy computation from overloaded MCU to gateway.
- **Fault recovery:** if device fails, reassign its automata to backup device.
- **Network optimization:** co-locate frequently communicating automata to reduce latency.

This requires:

1. Pause automaton, serialize state (including Lua variable values).
2. Transmit state snapshot to target device.
3. Resume execution on new device with identical state.

The YAML-based representation simplifies this: the automaton specification is platform-independent, and the Lua VM state is serializable.

2.5.4 Use cases for formal verification

Hierarchical composition enables modular verification:

- **Component verification:** prove atomic automata satisfy local properties (e.g., "actuator never exceeds safe range").
- **Compositional verification:** assume child properties hold, prove parent satisfies system-level requirements.
- **Deployment verification:** check that device mapping preserves timing constraints (e.g., "sensor-to-actuator latency < 50ms").

Example workflow:

1. Design automaton in IDE.
2. Export to UPPAAL timed automaton format.
3. Model check properties: reachability, deadlock freedom, timing bounds.

4. If verified, deploy to devices with confidence.
5. Use time-travel debugging to validate runtime behavior matches model.

This bridges the gap between formal methods (academic rigor) and practical IoT engineering (iterative prototyping).

2.6 IEC 61499

IEC 61499 is an international standard for distributed industrial control systems. It defines a model for "function blocks" that communicate via events and data, and a framework for deploying those blocks across devices [3].

In a typical IEC 61499 setup, applications are constructed as networks of blocks communicating through typed ports; those networks are then deployed onto a set of devices. The standard also defines concepts that matter for an IDE-driven workflow: deployable configurations, distribution decisions, and online changes.

2.6.1 Function blocks and event-driven execution

IEC 61499 function blocks separate *events* (control flow) from *data* (values). A function block has:

- Event inputs and outputs (control flow),
- Data inputs and outputs (sampled values),
- Internal algorithms (computation),
- An Execution Control Chart (ECC) that defines when algorithms execute in response to events.

The execution model works as follows:

1. An event arrives at an event input.
2. The ECC determines which algorithm(s) to execute based on the current state and the triggering event.
3. Algorithms read data from data inputs, perform computation, and write to data outputs.
4. The ECC may transition to a new state and emit output events.

This event-driven model encourages designs where computation is triggered by events and data is sampled at well-defined points, which is important for predictable behavior in distributed systems. Many engineering toolchains (including Eclipse 4diac) build on this: the IDE defines a network, a deployment configuration maps it to devices, and the runtime executes event-driven logic.

2.6.2 Service Interface Function Blocks (SIFBs)

IEC 61499 defines Service Interface Function Blocks as the mechanism for interacting with the outside world (hardware, network, timers). SIFBs abstract platform-specific details behind standard event/data interfaces.

For example:

- A PUBLISH SIFB sends data over a network when triggered by an event.
- A SUBSCRIBE SIFB receives network data and emits an event to notify the application.
- An E_CYCLE SIFB generates periodic events based on a timer.

This abstraction is relevant for the proposed architecture: the runtime must provide a similar abstraction layer (HAL for hardware, transport layer for communication) so that application logic remains portable across targets.

2.6.3 Deployment model and distribution

IEC 61499 distinguishes between:

- **Application model:** the network of function blocks (what the system does).
- **System model:** the set of devices and resources (where the system runs).
- **Distribution mapping:** assignment of function blocks to devices.

The standard supports online reconfiguration: function blocks can be created, deleted, or migrated between devices at runtime. This is a powerful concept for adaptive systems, but it requires careful runtime support (consistent state, connection management, synchronization).

In this repository, IEC 61499 serves primarily as an *inspiration* for modeling and deployment concepts; the project does not implement a full IEC 61499 runtime. Instead, it explores a smaller prototype with an explicit automata model and a simpler text format (YAML).

The key architectural takeaway is the overall toolchain shape: *model authoring* → *validation* → *deployment mapping* → *runtime execution* → *monitoring*. This thesis uses that workflow as a reference when defining the three-layer architecture in Chapter 3.

2.7 Visual Programming and Flow-Based Languages

Before examining specific tools, it's useful to understand the broader context of visual programming for automation and how different representational choices affect the engineering workflow.

2.7.1 Flow-based programming paradigm

Flow-based programming (FBP) treats applications as networks of processes communicating via data streams. Key concepts:

- **Nodes** (processes) perform computation.

- **Edges** (connections) carry data between nodes.
- **Ports** define typed interfaces for inputs and outputs.

FBP is particularly suited for event-driven and data-processing applications: the visual graph clearly shows data flow, and the asynchronous execution model maps naturally to distributed systems.

However, FBP graphs typically do not represent *state machines* or *timing constraints*. For control-intensive applications (state-dependent behavior, timing requirements), state machine notations (statecharts, automata) are more natural.

2.7.2 State machine vs. dataflow representations

The choice between state machines and dataflow graphs reflects different modeling priorities:

- **State machines:** emphasize control flow, modes of operation, and state-dependent transitions. Well-suited for modeling protocols, reactive behavior, and mode changes.
- **Dataflow graphs:** emphasize data transformation, processing pipelines, and parallel computation. Well-suited for signal processing, data analytics, and integration tasks.

IEC 61499 combines both: function blocks have internal state machines (ECCs) and external dataflow connections. This hybrid approach provides expressiveness at the cost of complexity: developers must understand both execution models and their interaction.

The proposed approach uses automata as the primary abstraction, treating dataflow as secondary (variables passed between automata, rather than explicit dataflow edges). This is a deliberate simplification to keep the runtime model tractable.

2.8 Debugging Paradigms and Time-Travel Debugging

2.8.1 Static vs. dynamic debugging

Debugging approaches can be broadly categorized:

- **Static analysis:** detecting errors by analyzing code without executing it (type checking, linting, model checking).
- **Dynamic debugging:** inspecting behavior during execution (breakpoints, logging, profiling).

For distributed embedded systems, static analysis alone is insufficient: timing-dependent bugs, concurrency issues, and hardware interactions only manifest at runtime. Traditional dynamic debugging (pausing execution at a breakpoint) is problematic for real-time systems: pausing violates timing constraints and may mask or alter the bug.

2.8.2 Trace-based and time-travel debugging

Time-travel debugging (also called omniscient debugging or reversible debugging) means that execution is recorded as a sequence of snapshots (or events), and the developer can navigate backward/forward through that history to inspect state.

Key characteristics:

- **Non-intrusive:** recording happens passively; execution is not interrupted.
- **Post-mortem analysis:** debugging happens after the run completes (or after a failure is detected).
- **Deterministic replay:** given the same initial state and input sequence, the system reproduces the same execution.

Enabling time-travel debugging requires:

1. **Deterministic semantics:** the system's behavior must be reproducible (no uncontrolled nondeterminism from uninitialized memory, timing races, etc.).
2. **State capture:** periodic snapshots of system state (or event logs sufficient to reconstruct state).
3. **Input recording:** all external inputs (network messages, sensor readings, user commands) must be logged.
4. **Efficient storage:** full snapshots are expensive; delta encoding or event sourcing can reduce overhead.

In this repository, the IDE demonstrates the *user-facing surfaces* of this idea using simulated data (e.g., snapshots and device state changes). Nevertheless, this concept influences the design: deterministic semantics and reproducible traces are explicit goals because they enable replay.

2.8.3 Application to distributed control systems

For distributed automation systems, time-travel debugging is particularly valuable:

- Bugs often arise from timing interactions between distributed components.
- Reproducing failures in a lab setting is difficult (requires exact sensor inputs, network timing, etc.).
- Post-deployment debugging on embedded devices is constrained by limited resources and lack of interactive debugging tools.

By recording execution traces and replaying them in a simulator or IDE, developers can:

- Inspect the state of all components at any point in time.
- Step backward to understand the sequence of events leading to a failure.
- Test hypothetical fixes by replaying with modified logic.

This is why the assignment emphasizes time-travel debugging as a key feature: it enables a fundamentally different debugging workflow for distributed embedded systems.

2.9 Dynamic Reconfiguration and Online Changes

One of the key advantages of event-driven, block-based architectures is the potential for *dynamic reconfiguration*: modifying the system's behavior while it continues to run. This capability is particularly valuable for distributed IoT and automation systems that must adapt to changing requirements without service interruption.

2.9.1 Motivation and use cases

Dynamic reconfiguration addresses several practical needs:

- **Bug fixes without downtime:** patch a faulty state transition or guard condition without halting the entire system.
- **Adaptive behavior:** deploy new control strategies in response to changing environmental conditions or learned patterns.
- **Incremental deployment:** roll out updates to a subset of devices for testing before system-wide deployment.
- **A/B testing:** run multiple versions of an automaton simultaneously to compare performance.

2.9.2 IEC 61499 online reconfiguration model

IEC 61499 defines explicit support for online reconfiguration through management commands:

- CREATE: instantiate a new function block.
- DELETE: remove a function block.
- START/STOP: control execution of function blocks.
- WRITE: update parameters or connections.

The standard requires that these operations maintain system consistency: connections must be valid, data types must match, and the system must reach a coherent state after each operation.

2.9.3 Aetherium approach: YAML hot-reload

The YAML-based representation (see Section 2.10) enables a simpler reconfiguration model:

1. The automaton specification is loaded from a YAML file.
2. The IDE or deployment tool can push updated YAML to a running device.
3. The runtime validates the new specification (type checking, connectivity, guard syntax).
4. If validation succeeds, the runtime transitions from the old automaton to the new one, preserving state where possible.

Challenges for reconfiguration:

- **State continuity:** if the new automaton renames or removes states, what happens to a device currently in that state?
- **Variable migration:** how are variable values transferred from old to new automaton?

- **Consistency:** in a distributed system, how do we ensure all devices transition to the new version consistently?

Proposed strategies (documented but not yet implemented):

- **State mapping:** the YAML can include an optional migration section that maps old state IDs to new ones.
- **Graceful transition:** the runtime completes current state transitions before loading the new automaton.
- **Versioning:** each YAML has a version number; the runtime can refuse incompatible updates or apply migration rules.

This design anticipates future work on distributed consensus and coordinated reconfiguration across multiple devices.

2.10 YAML-Based Modeling and Specification

The project uses YAML as the primary authoring format instead of XML (IEC 61499) or proprietary formats. YAML provides human readability, version control compatibility, and extensibility while remaining lightweight.

2.10.1 Core structure

The format defines three sections: `version` (spec evolution), `config` (metadata, inline vs. folder layout), and `automata` (states, transitions, inputs/outputs, variables).

Example:

```
version: 0.0.1
config:
  name: controller
  type: inline
automata:
  initial_state: Idle
  states:
    Idle:
      code: |  -- Lua code here
        if value("input") > threshold then
          setVal("output", 1)
        end
  transitions:
    ToActive:
      from: Idle
      to: Active
      condition: value("input") > 100
      weight: 0.9  -- probabilistic
```

2.10.2 Folder layout for large projects

For maintainability, code can live in separate `.lua` files (`State1.lua`, `Transition1.lua`) with standard IDE support and modular testing. This enables syntax highlighting, unit testing per state, and cleaner Git diffs.

2.10.3 Lua runtime API

Scripts access a minimal deterministic API: `check(name)`, `value(name)`, `setVal(name, val)`, `now()`, `rand()`, `log(level, msg)`. Determinism is critical for time-travel debugging: identical inputs yield identical outputs.

(Full API specification in Appendix **??**.)

2.10.4 Validation

Validation occurs at four levels: YAML syntax, schema (required fields), semantics (state existence, connectivity), and Lua code syntax. The C++ engine implements basic validation; future IDE integration will provide real-time error highlighting.

2.11 ROS2 and Micro-ROS

ROS (Robot Operating System) originated as a middleware framework for robotics research; ROS2 is a redesign targeting real-time and embedded systems [5]. Micro-ROS extends ROS2 to microcontrollers by providing a lightweight client library compatible with DDS-XRCE (a constrained-environment variant of DDS) [4].

2.11.1 ROS2 architecture and DDS

ROS2 uses DDS (Data Distribution Service) as its underlying communication middleware. DDS provides:

- **Publish-subscribe messaging:** nodes publish data on topics; subscribers receive updates.
- **Quality of Service (QoS) policies:** reliability (best-effort vs. reliable), durability (transient vs. volatile), deadline constraints, etc.
- **Discovery:** automatic detection of publishers/subscribers on the network.

This architecture enables loose coupling: nodes don't need to know each other's network addresses; they communicate through named topics managed by the DDS middleware.

2.11.2 Micro-ROS and resource constraints

Micro-ROS adapts ROS2 for microcontrollers by:

- Using DDS-XRCE, a lightweight profile of DDS designed for constrained devices.
- Providing an agent (running on a gateway or companion computer) that bridges DDS-XRCE to full DDS.
- Offering a reduced API footprint compatible with RTOS environments (FreeRTOS, Zephyr, etc.).

The agent pattern is relevant for this thesis: it separates resource-constrained edge devices from the full middleware complexity. Edge devices run minimal client logic and communicate with an agent over a simple protocol; the agent handles integration with the broader distributed system.

2.11.3 Relevance to this project

The relevance of ROS2/Micro-ROS here is primarily architectural: they provide a reference for how a heterogeneous distributed system can share a message schema and still run on both Linux and MCUs. In the current prototype, the equivalent "control plane" concept is implemented with Phoenix Channels (Chapter 4), while device-side protocols (MQTT/UDP/DDE, DDS, DDS-XRCE) remain future work.

The key takeaway is the overall pattern:

- Edge devices communicate with a gateway/agent via a constrained protocol.
- The gateway integrates with a richer ecosystem (web UI, monitoring, data storage).
- The system supports heterogeneous platforms (MCUs, Linux, cloud).

Because ROS2/Micro-ROS are broad ecosystems, this thesis treats them mainly as a reference point for middleware concerns (message schema, QoS, discovery) rather than as a direct dependency.

2.12 Comparison: Node-RED vs. 4diac/FORTE vs. Aetherium Automata

This section positions Aetherium Automata relative to two established tools: Node-RED (pragmatic IoT integration) and Eclipse 4diac/FORTE (IEC 61499 industrial automation).

2.12.1 Key differentiators

Node-RED: flow-based visual editor for rapid IoT prototyping. Strengths: extensive ecosystem (thousands of nodes), low barrier to entry, runs anywhere (RPi to cloud). Limitations: no formal semantics, weak determinism (JavaScript event loop), no time-travel debugging, JSON representation. Relevance: demonstrates viability of visual event-driven programming; Aetherium borrows IDE approach but adds formal semantics.

4diac/FORTE: IEC 61499-compliant toolchain for industrial automation. Strengths: standards compliance, event-driven execution (ECCs), real-time capable, explicit deployment model, online reconfiguration. Limitations: high complexity, limited verification (no model checking), no time-travel debugging, verbose XML representation. Relevance: provides reference workflow (model → deploy → monitor); Aetherium adopts structure but uses lighter YAML format and targets debugging ergonomics.

Aetherium Automata: formal control with modern tooling. Design goals: DEVS/timed automata semantics, YAML representation, time-travel debugging, hierarchical composition, fuzzy/probabilistic transitions, heterogeneous deployment (MCU to cloud). Trade-offs: prototype maturity, limited ecosystem, non-standard format (no IEC 61499 interop), planned verification integration.

2.12.2 Summary by dimension

- **Modeling:** Node-RED (dataflow), 4diac (function blocks + ECCs), Aetherium (state machines + fuzzy/probabilistic).
- **Semantics:** Node-RED (implicit timing), 4diac (event-driven), Aetherium (DEVS/TA determinism).
- **Debugging:** Node-RED (live inspection), 4diac (monitoring), Aetherium (time-travel replay).
- **Deployment:** Node-RED (single runtime), 4diac (device mapping), Aetherium (hierarchical + migration).
- **Format:** Node-RED (JSON), 4diac (XML), Aetherium (YAML).
- **Verification:** Node-RED (none), 4diac (syntax checking), Aetherium (designed for UPPAAL/PRISM export).

Aetherium occupies a niche: formal semantics for verification + modern tooling for rapid iteration, targeting research and safety-critical IoT rather than general integration or heavy industrial use.

Chapter 3

Requirements and System Design

Chapter 2 motivates two core design pressures: (i) explicit semantics for time and events (DEVS, timed automata), and (ii) a practical engineering toolchain (IEC 61499, 4diac/-FORTE, and lessons from Node-RED). The system design below translates those pressures into concrete architectural decisions: representation format, layering, and the control-plane responsibilities.

3.1 Assignment requirements vs. repository status

Table 3.1 maps the assignment requirements to (a) the architectural approach taken in this repository and (b) what the current prototype demonstrates.

Requirement (condensed)	**Architecture / current demonstrator**
Study DEVS, IEC 61499, Node-RED, 4diac/-FORTE, ROS2/Micro-ROS	Background + positioning (this thesis) and design docs
RTE for interpreted communicating timed automata	Model representation + validation pipeline + architecture for runtime integration
GPIO/I2C access via HAL	Defined as a layer boundary (not exercised in current demo)
Communication via MQTT/UDP/DDE	Defined as a transport boundary; current control plane uses WebSockets
IDE as web application	Implemented as Electron (web UI stack) to validate UX and integration quickly
Visual modeling + code generation + deployment	Visual editor prototype; representation designed for generation workflows
Monitoring, time-travel debugging, dynamic reconfiguration	IDE concepts + protocol/service scaffolding; traces are a design requirement

Table 3.1: Implemented vs. planned features as of the repository state.

3.2 High-level architecture

The repository is organized into three layers:

1. **IDE:** Electron+React application with a visual automata editor and gateway panel.
2. **Gateway + Server:** Elixir/Phoenix services communicating via Phoenix Channels.
3. **Engine (RTE prototype):** a C++ binary that loads YAML automata definitions into internal structures.

In this architecture, the gateway layer is the *coordination point* between user intent (IDE) and distributed execution. It separates (a) interactive UI concerns from (b) device-facing execution concerns. In the current repository state, the gateway already demonstrates this separation by handling IDE connectivity, session/log events, and aggregating device lists reported by server agents.

Current prototype architecture (schematic)

IDE (Electron/React) ↔ **Gateway (Phoenix Channels)** ↔ **Server agent (PhoenixClient)**

Engine (C++ YAML parser/validator) is currently built and run separately as a CLI tool.

Figure 3.1: Schematic view of the implemented communication/control plane and the current engine prototype boundary.

3.3 Automata representation

Automata are specified in YAML. The documentation describes two YAML styles (list-of-singletons and canonical mapping form), and two code layouts (inline and folder-based) [1].

3.3.1 Why YAML

The repository chooses YAML as a human-readable interchange format that supports:

- manual editing and review,
- embedding code snippets (as strings) during prototyping,
- a straightforward validation story (required keys, typed variable declarations).

This is a deliberate contrast to tool-specific representations: Node-RED uses JSON flows, and IEC 61499 toolchains often use specialized artifacts. YAML is not intended to replace those ecosystems; it is used here to support a lightweight, repository-first workflow.

The current C++ implementation expects the canonical mapping form and validates the presence of top-level keys `version`, `config`, and `automata`.

Chapter 4

Implementation

This chapter describes the implementation as present in the repository.

4.1 C++ engine (RTE prototype)

The engine is built with CMake and produces a host binary (macOS/Linux). At this stage it acts as a *model loader and validator* that makes the representation concrete and testable. The implemented functionality is:

- CLI option parsing (help/version/validate/run flags).
- YAML parsing using RapidYAML into internal model structures (states, transitions, variables).
- Basic schema validation: required top-level keys are checked.

The execution loop is currently a placeholder: it does not evaluate transition guards, advance time, or execute state bodies.

4.1.1 In-memory model (what exists in code)

The repository defines a minimal set of C++ types that represent the automata structure:

- `Automata`: vectors of `State`, `Transition`, and global `Variables`; plus metadata (version, name, type).
- `State`: lists of input/output/variable declarations and a `Code` body (stored as a string).
- `Transition`: pointers to `State` objects (`from`/`to`) and a `condition` code block.

Parsing links transitions to states by matching state names. This keeps the prototype simple, but it is currently $O(|S| \cdot |T|)$ and is marked in code as a candidate for optimization.

4.1.2 YAML-to-object mapping (illustrative pseudocode)

```
// Conceptual sketch based on the current parser implementation
Automata a;
a.version = root["version"];
```

```
a.name = root["config"]["name"];

for (stateNode : root["automata"]["states"].children())
  a.states.push_back(parseState(stateNode));

for (trNode : root["automata"]["transitions"].children()) {
  Transition t = parseTransition(trNode);
  t.from = findStateByName(a.states, trNode["from"]);
  t.to = findStateByName(a.states, trNode["to"]);
  a.transitions.push_back(t);
}
```

Importantly, `Code` blocks are currently parsed as strings and stored; they are not executed.

4.1.3 Validation behavior

The validator currently checks the presence of required top-level keys (`version`, `config`, `automata`). The checks are implemented with RapidYAML's `RYML_CHECK`, which is appropriate for a prototype validator but motivates a design requirement for IDE-facing diagnostics.

4.2 Gateway and server infrastructure (Elixir/Phoenix)

The gateway exposes Phoenix Channels for two client types:

- the IDE joins `gateway:control` and can call `ping`, `list_devices`, `list_servers`, and `restart_device`,
- the server joins `server:gateway` and periodically sends `heartbeat` and `device_update`.

The gateway aggregates device lists from connected servers and broadcasts updates to the IDE. Authentication is currently based on development tokens (hard-coded shared secrets), which is sufficient for a prototype but not for production.

4.2.1 Message flow (implemented)

The current message flow is intentionally minimal and can be summarized as follows:

- **IDE → gateway:** join `gateway:control` with a development token; optionally call `list_devices` and `list_servers`.
- **Server agent → gateway:** join `server:gateway` with `server_id` and token; periodically send `heartbeat`; publish `device_update` with a device list.
- **Gateway → IDE:** push `device_list` and `server_list`; broadcast `devices_updated` and `alert/log` events.

4.3 IDE prototype (Electron + React)

The IDE is implemented as an Electron application using React. It contains:

Sequence sketch (device list)

Server agent → gateway: `device_update(devices)`
Gateway → IDE: `devices_updated(...)` and `device_list(aggregate)`

Figure 4.1: A simplified message-flow sketch of the implemented device aggregation path.

- a visual automata editor implemented with ReactFlow,
- a code editor based on Monaco,
- a gateway panel capable of connecting to the Phoenix gateway and showing devices/servers.

IDE layout (schematic)

Left: Project/navigation + device/server list (gateway panel)
Center: Visual editor canvas (states as nodes, transitions as edges)
Right/bottom: Code editor (Monaco) + logs/alerts + time-travel panel (currently mock)

Figure 4.2: A schematic view of the IDE panels to complement the text description.

Many advanced features required by the assignment (deployment, monitoring, time-travel debugging, dynamic reconfiguration) exist as UI panels and service interfaces but are backed by mock implementations.

In this context, *mock* means that the UI is wired to a local simulated service that generates example devices, automata, and execution snapshots. This allows the front-end workflow (panels, state rendering, navigation) to be tested before a real runtime and trace pipeline exist.

4.3.1 Why Electron and Phoenix (prototype rationale)

The choice of Electron provides a fast path to a desktop IDE while reusing web technologies (React, ReactFlow, Monaco). This partially meets the "web IDE" requirement: the UI is web-based, but packaged as a desktop application.

Phoenix Channels provide a simple WebSocket-based control plane for the prototype. The TypeScript service connects to `ws://<host>:<port>/socket` and joins the `gateway:control` topic. This provides an immediately testable integration point between UI and backend and demonstrates the intended separation between IDE and distributed execution.

Chapter 5

Demonstration

This repository contains a minimal demonstrator that is consistent with the current implementation. It intentionally demonstrates the parts that are already real and testable (parsing/validation, gateway aggregation, IDE connectivity), and separates planned end-to-end features as future work.

5.1 Runnable demonstrator (current)

1. Build the C++ engine and run `-validate` on the provided YAML examples.
2. Start the Phoenix gateway and server to see device list aggregation in the IDE.
3. Use the IDE to connect to the gateway and inspect device/server lists and logs.

5.2 Planned end-to-end case study (not implemented yet)

For the final thesis, a complete scenario should demonstrate model authoring → deployment → runtime execution → monitoring. One suitable candidate is a small distributed controller such as a traffic-light intersection or a sensor/actuator loop with a gateway bridge. At the current stage, this section is a design placeholder: the repository does not yet execute automata or deploy them to devices.

1. Build the C++ engine and run `-validate` on the provided YAML examples.
2. Start the Phoenix gateway and server to see device list aggregation in the IDE.
3. Use the IDE to visually edit an automaton and explore mock time-travel/debug panels.

The demonstrator is intentionally limited: it does not yet execute automata on devices.

Chapter 6

Discussion and Future Work

This chapter focuses on architectural trade-offs and how the design positions the project relative to existing solutions.

6.1 Architectural trade-offs

- **Representation trade-off:** choosing YAML prioritizes readability and repository workflows over strict standard compatibility.
- **Layering trade-off:** separating IDE, gateway, and execution components improves modularity and supports distributed operation, but requires well-defined protocols.
- **Debugging trade-off:** time-travel debugging requires deterministic semantics and traceability; this drives constraints on the execution model and on what information must be represented.

6.2 Positioning relative to Node-RED and 4diac

Node-RED and 4diac/FORTE are mature ecosystems. The purpose of Aetherium Automata is not feature parity; it is to explore an architecture that combines a lightweight, inspectable representation with an IDE workflow and a control plane suitable for distributed systems.

Chapter 7

Conclusion

This thesis presents the design and the current implementation status of Aetherium Automata. The repository contains a working control-plane prototype (IDE $\leftrightarrow$ gateway) and a C++ YAML-to-model pipeline. The core runtime features required for distributed interpreted timed automata are not implemented yet; therefore the main result is an honest, test-backed baseline for continuing development.

Bibliography

[1] Aetherium Automata Project. *Aetherium Automata YAML Format (repository documentation)* online. 2025. Available at: https://github.com/. [cit. 2026-01-17]. Repository-local documentation: docs/Automata_YAML_Spec.md (URL placeholder if unpublished).

[2] Alur, R. and Dill, D. L. *A Theory of Timed Automata* journal article, Theoretical Computer Science 126(2):183–235. 1994. Available at: https://doi.org/10.1016/0304-3975(94)90010-8. [cit. 2026-01-17].

[3] International Electrotechnical Commission. *IEC 61499-1: Function blocks – Part 1: Architecture* standard. 2012. Available at: https://webstore.iec.ch/searchform?q=61499-1. [cit. 2026-01-17]. IEC standard. Access may require purchase.

[4] micro-ROS community. *Micro-ROS* online. 2026. Available at: https://micro.ros.org/. [cit. 2026-01-17].

[5] Open Robotics. *ROS 2 Documentation* online. 2026. Available at: https://docs.ros.org/en/rolling/. [cit. 2026-01-17].

[6] Zeigler, B. P.; Praehofer, H. and Kim, T. G. *Theory of Modeling and Simulation: Integrating Discrete Event and Continuous Complex Dynamic Systems*. 2nd ed. Academic Press, 2000. ISBN 978-0127784557.

Appendix A

Repository Overview

This appendix lists the main repository components relevant to the thesis.

Engine (C++) `src/engine/` contains the CLI, YAML parser, and validator.

Transports (C++) `src/transport/` contains transport interfaces and stubs.

Gateway (Elixir/Phoenix) `src/gateway/aetherium_gateway/` contains the Phoenix application and channels.

Server (Elixir) `src/server/aetherium_server/` contains the gateway-connecting agent.

IDE (Electron/React) `src/ide/` contains the Electron application.

Docs `docs/` contains the YAML specification, architecture notes, and protocol drafts.

Tests `tests/` contains Python-based CLI tests and a CLI behavior spec.

Appendix B

Build and Run Instructions

The following commands reflect the repository structure at the time of writing. They are provided as guidance; exact dependencies (compilers, Elixir/Node versions) depend on the local environment.

B.1 Engine (C++)

```
cmake -S . -B build
cmake --build build
./build/aetherium_engine --help
./build/aetherium_engine --version
./build/aetherium_engine --validate example/automata/automata-yaml-examples/one-state-a
```

B.2 Gateway (Elixir/Phoenix)

```
cd src/gateway/aetherium_gateway
mix deps.get
mix phx.server
```

B.3 Server (Elixir agent)

```
cd src/server/aetherium_server
mix deps.get
mix run --no-halt
```

B.4 IDE (Electron + React)

```
cd src/ide
npm install
npm run dev
```

Appendix C

Notes on Current Limitations

- The IDE is implemented as an Electron desktop application, not as a browser-hosted web application.
- Engine validation may terminate the process on malformed YAML due to assertion-style checks.
- Transport implementations (MQTT/UDP/DDE) and hardware access (GPIO/I2C) are not implemented.
- Execution semantics for timed automata and time-travel debugging are currently not implemented in the runtime; the IDE includes mock UI for these features.